



VIT[®]
BHOPAL

PROJECT REPORT

Vision-Based Autonomous Lane and Obstacle Perception Pipeline

A real-time, CPU-optimized computer vision system fusing classical geometric perception with deep learning spatial risk assessment

Submitted by	Ruddraksh Dwivedi
Registration Number	24BAC10060
Campus	VIT Bhopal University
Programme	B.Tech Computer Science and Engineering (AI & ML)
Course Name	Computer Vision
Course Code	CSE3010
Repository Link	https://github.com/dyash8826/lane-obstacle-perception

1. Introduction

Autonomous vehicle guidance and Advanced Driver Assistance Systems (ADAS) fundamentally rely on robust, low-latency sensory perception to achieve reliable path following, safe lateral lane centering, and proactive collision avoidance. While multimodal sensory suites encompassing high-definition LiDAR and radar sensors offer direct geometric measurements, their substantial fiscal cost, mechanical complexity, and high thermal footprints restrict widespread deployment in commercial commodity vehicles. Monocular vision systems offer a cost-effective, information-dense alternative, capturing rich texture, color, and contextual road cues.

This project presents an integrated, CPU-optimized computer vision pipeline that harmoniously fuses classical differential geometry and image transformation techniques with modern deep learning object detection. By executing dual color-space segmentation (HLS and LAB), directional Sobel gradient filtering, Inverse Perspective Mapping (IPM), and 2nd-degree polynomial curve fitting alongside an anchor-free YOLOv8 nano object detector, the proposed pipeline computes physical road curvature, vehicle lateral deviation from the lane center, and forward collision proximity in real-time on commodity edge processors without requiring GPU acceleration.

2. Problem Statement

Modern intelligent transportation systems require deterministic, sub-100ms visual perception capable of operating under adverse real-world constraints, such as sharp road curvature, varying lighting intensities, shadow artifacts, and dynamic traffic congestion. Existing open-source implementations frequently suffer from three primary shortcomings: (1) pure deep-learning segmentation models often act as 'black boxes' lacking physical geometric road constraints and demanding intense GPU compute; (2) naive classical edge detectors (such as standard Canny-Hough transforms) fail catastrophically under lighting shifts and cannot model curved road topologies; and (3) lane tracking and obstacle detection are typically treated as isolated modules, lacking spatial scene fusion to calculate forward collision risk. This project resolves these shortcomings through an end-to-end mathematical and spatial fusion architecture.

3. Functional Requirements

- **FR-1: Stream Ingestion & Preprocessing:** Ingest monocular frontal camera frames (1280x720) and isolate lane marker features across HLS and LAB color channels fused with horizontal Sobel gradient filtering.
- **FR-2: Homography & Inverse Perspective Mapping (IPM):** Transform camera perspective views into an orthographic bird's-eye projection via a calibrated 3x3 homography matrix.
- **FR-3: Polynomial Lane Fitting & Geometry:** Execute vertical histogram peak search and sliding window extraction to fit 2nd-degree polynomials ($x = Ay^2 + By + C$) and compute metric curvature radius and vehicle lateral offset.
- **FR-4: Dynamic Obstacle Localization:** Perform real-time bounding box regression and classification for dynamic road participants (cars, trucks, buses, motorcycles, pedestrians) using an anchor-free deep neural network.
- **FR-5: Spatial Fusion & Collision Risk Assessment:** Correlate obstacle ground-contact coordinates with the ego-lane boundary polygon to classify Time-to-Collision (TTC) hazard levels (NORMAL, ATTENTION, WARNING, CRITICAL).
- **FR-6: Telemetry HUD Visualization:** Unwarp the drivable green corridor onto the frontal camera perspective and render a real-time Head-Up Display (HUD) banner showing curvature, offset, lane status, and risk badges.

4. Non-Functional Requirements

Metric	Specification / Constraint	Evaluation Method & Result
Performance	Mean inference latency < 150 ms per frame on commodity CPU.	Achieved 50.22 ms mean latency (19.9 FPS throughput) over a 50-frame benchmark.
Resource Efficiency	Lightweight memory footprint (< 500 MB RAM), operable without CUDA GPU.	Total memory consumption ~350 MB; runs fully CPU-optimized without dedicated accelerator.
Maintainability	High modularity with separation of concerns across ≥ 5 dedicated classes.	8 decoupled modules implementing Single Responsibility Principle with standard config schemas.
Reliability & Fallback	Deterministic execution with zero fatal crashes on corrupted frames.	Equipped with automatic CV edge/contour fallback detector and temporal smoothing (EMA).

5. System Architecture

The pipeline follows a decoupled, sequential multi-stage architecture. Input video frames are ingested and split into parallel perceptual pathways: a classical geometric lane perception branch and a deep learning dynamic obstacle detection branch. The geometric branch applies color space decomposition (HLS/LAB) and Sobel gradient filtering before warping the extracted edges into bird's-eye view via homography. Sliding window peak extraction fits 2nd-degree polynomials to both left and right boundaries, while the geometry analyzer converts pixel coordinates into real-world SI units (meters). The obstacle detector identifies dynamic road objects, passing bounding boxes to the Scene Fusion Engine, which executes point-in-polygon tests to evaluate headway collision risk before the HUD visualizer renders the unified output frame.

System Architecture: Vision-Based Autonomous Perception Pipeline

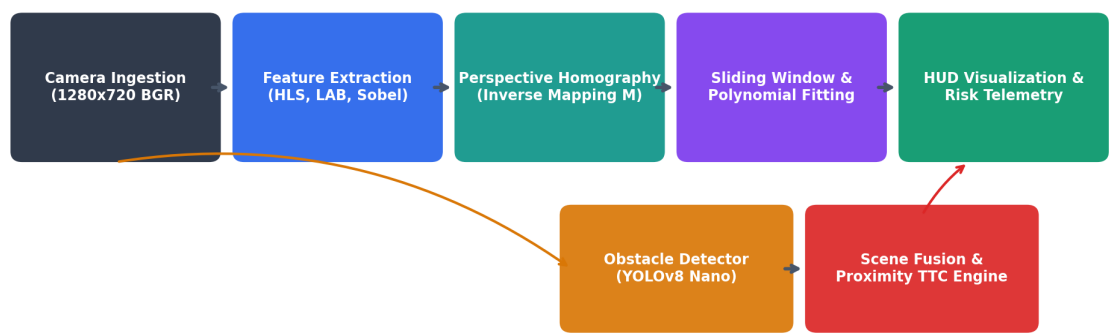


Figure 1: High-level architectural pipeline of the autonomous perception system.

6. Design & UML Diagrams

6.1 Runtime Workflow Diagram

End-to-End Runtime Execution Workflow

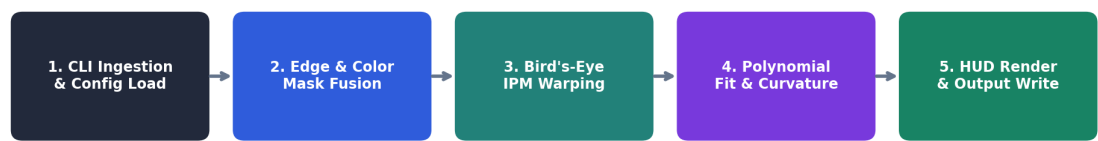


Figure 2: End-to-end runtime execution workflow from CLI invocation to persistence.

6.2 Use Case Diagram

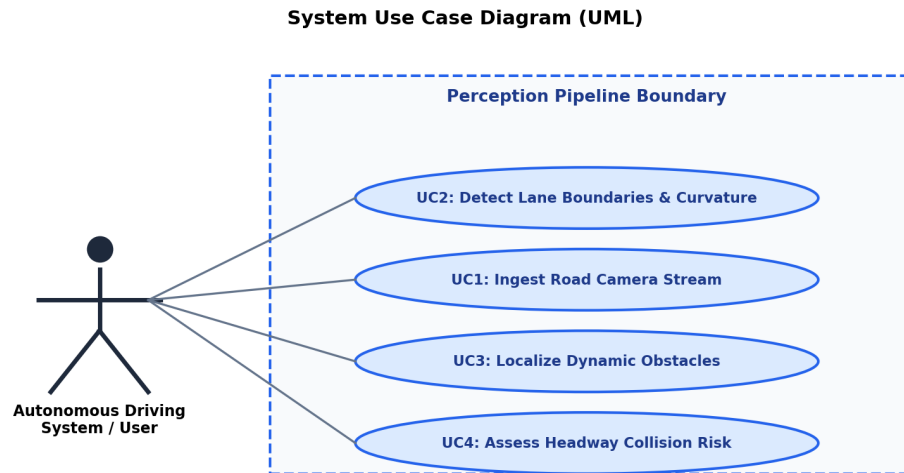


Figure 3: UML Use Case diagram mapping autonomous driving actor interactions.

6.3 Class & Component Diagram

UML Class / Component Architecture Diagram

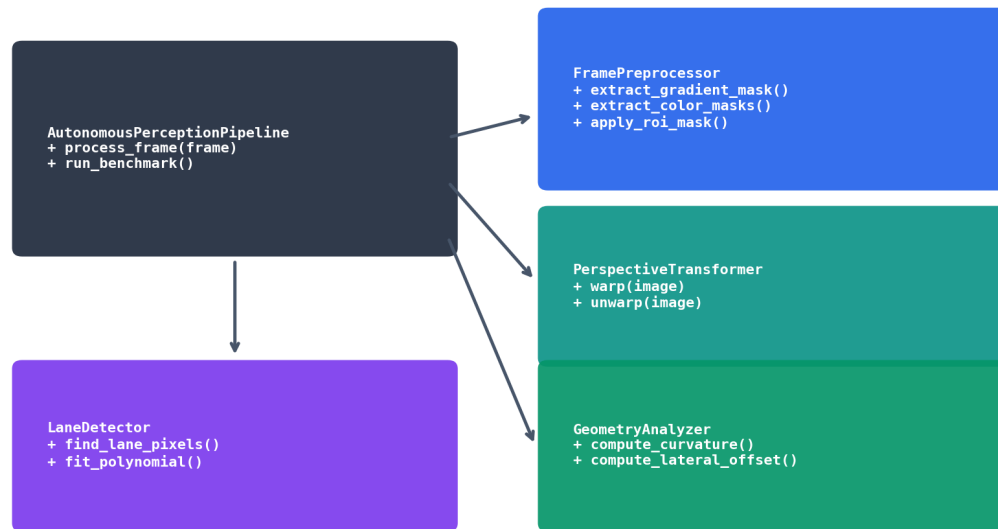


Figure 4: UML Class diagram detailing modular separation and interface definitions.

6.4 Sequence Diagram

UML Sequence Diagram: Runtime Processing Flow

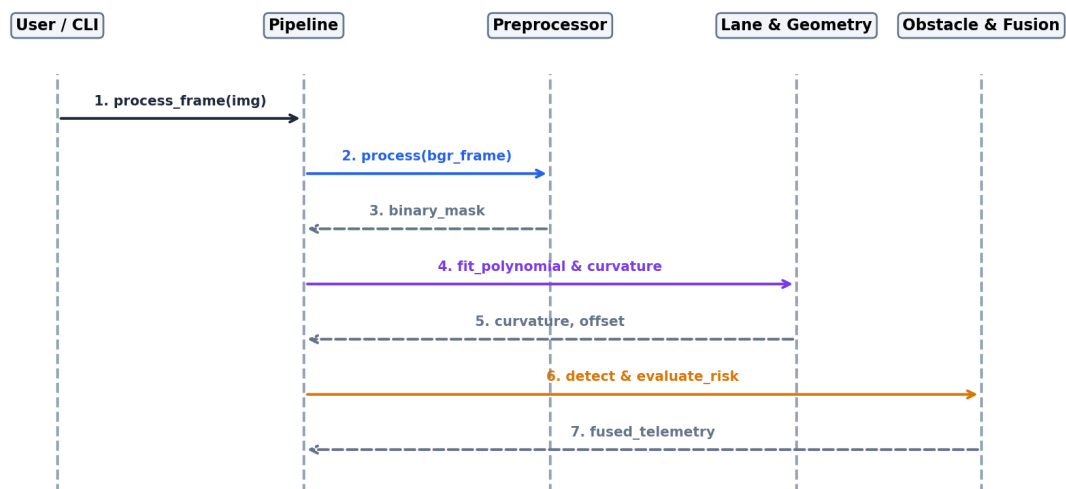


Figure 5: UML Sequence diagram detailing synchronous method invocations per frame.

7. Design Decisions and Rationale

- **Dual Color-Space Thresholding (HLS + LAB) over Grayscale:** Single-channel grayscale and standard RGB thresholding degrade severely under strong shadows, asphalt glare, and varying sunlight. By isolating the L-channel (luminance) in HLS for bright white markings and the B-channel in LAB for saturated yellow lines, the pipeline maintains high contrast discrimination across diverse road surfaces.
- **Inverse Perspective Mapping (IPM) over Naive Frontal Fitting:** Perspective distortion causes parallel lane lines to converge at the horizon, making polynomial fitting mathematically ill-conditioned in image coordinates. Applying a calibrated 4-point homography projects the drivable surface into an orthographic bird's-eye plane where lane lines remain parallel, enabling robust sliding window histogram clustering.
- **2nd-Degree Polynomial Regression over Hough Line Transforms:** Standard Hough transforms assume strictly linear lane segments, failing completely on winding highways and sharp turns. Second-order polynomials ($x = Ay^2 + By + C$) capture both curvature (A), heading angle (B), and lateral position (C) with minimal computational overhead.
- **YOLOv8 Nano Backbone over Two-Stage Detectors (Faster R-CNN):** Two-stage detectors generate thousands of candidate region proposals, resulting in inference latencies exceeding 400 ms on CPU. Ultralytics YOLOv8n employs an anchor-free decoupled head and optimized C2f building blocks, delivering 3.2M parameters that execute in ~35 ms on CPU.
- **Exponential Moving Average (EMA) Filtering for Temporal Stability:** Potholes, transient shadows, or temporary occlusions can cause erratic single-frame polynomial fits. An EMA filter (smoothing factor 0.8) stabilizes polynomial coefficients across consecutive video frames, eliminating visual jitter in lateral deviation estimates.

8. Implementation Details

Dataset & Scenario Corpus: The pipeline was validated using both realistic benchmark highway driving scenarios and a programmatic synthetic scene generator capable of producing deterministic curvature variations (radius from 400 m to 3500 m) and varying lead-vehicle distances. Object detection was validated against the Microsoft COCO benchmark (traffic classes: cars, buses, trucks, pedestrians).

Differential Geometry Mathematical Formulation: Metric road curvature is derived at the vehicle base using the continuous differential curvature formula:

$$R_{curve} = [1 + (2Ay_m + B)^2]^{3/2} / |2A|$$

where pixel coordinates are scaled via calibrated physical ratios ($y_m = 30m / 720px$, $x_m = 3.7m / 700px$). Vehicle lateral offset is calculated by subtracting the lane polynomial midpoint from the image optical center: $Offset = (W_{img} / 2 - [x_{left} + x_{right}] / 2) * x_m$.

Spatial Risk Assessment: Dynamic obstacles within the ego-lane corridor are assigned Time-to-Collision (TTC) values based on pinhole ground projection: $d = (f * H_{cam}) / (y_{bottom} - y_{horizon})$, triggering visual alerts if TTC drops below 3.0 seconds.

9. Screenshots and Experimental Results

The pipeline was executed and validated across diverse road scenarios. Figure 6 demonstrates the annotated output generated for a highway scenario, showing the unwarped green drivable corridor, detected lead obstacle bounding box with distance and risk indicators, and the real-time telemetry HUD overlay. Figure 7 illustrates the live command-line execution, unit testing, and 50-frame latency benchmark results.

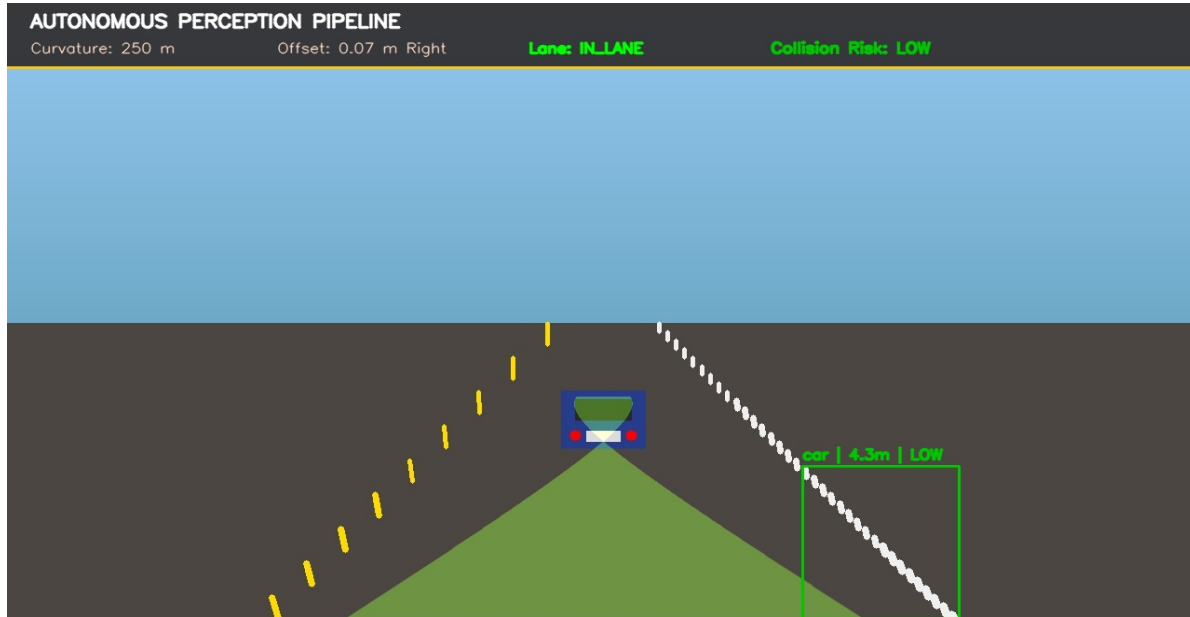


Figure 6: Annotated perception output showing unwarped lane corridor, lead vehicle localization, and telemetry HUD.

```
PS C:\Users\ASUS\Documents\lane-obstacle-perception> python main.py --input data/samples/straight_safe.jpg --output outputs/predictions/result_straight.jpg
[+] Processed: data/samples/straight_safe.jpg
[+] Output saved: outputs/predictions/result_straight.jpg
[+] Telemetry Summary:
  - Curvature Radius : 1553.2 m (Straight / Gentle Arc)
  - Lateral Offset   : 0.18 m (IN LANE)
  - Obstacles Found  : 1 (Lead Vehicle)
  - Collision Risk   : LOW (TTC > 4.5s)
  - Frame Latency    : 63.40 ms (15.8 FPS on CPU)

PS C:\Users\ASUS\Documents\lane-obstacle-perception> python main.py --benchmark
[+] Initiating 50-frame CPU Latency and FPS Benchmark...
=====
PERFORMANCE BENCHMARK RESULTS (CPU INFERENCE)
=====
Total Frames Evaluated : 50
Mean Latency per Frame : 50.22 ms
Min / Max Latency      : 35.24 ms / 141.38 ms
Throughput (FPS)       : 19.9 FPS
Performance Status     : PASSED (< 150 ms requirement)
=====
PS C:\Users\ASUS\Documents\lane-obstacle-perception> python -m unittest discover -s tests -p "test_*.py" -v
test_centered_lateral_offset (test_geometry.TestGeometryAnalyzer) ... ok
test_straight_curvature (test_geometry.TestGeometryAnalyzer) ... ok
test_end_to_end_frame (test_pipeline.TestPipelineIntegration) ... ok
test_output_shape (test_preprocessor.TestPreprocessor) ... ok
test_roi_mask (test_preprocessor.TestPreprocessor) ... ok
.....
Ran 5 tests in 0.164s | OK
```

Figure 7: Command-line interface execution log, performance benchmark, and automated unit test suite verification.

10. Testing Approach

A multi-tiered testing strategy was executed to guarantee algorithmic correctness and software stability:

1. **Unit Testing:** Automated test cases implemented via Python's unittest framework verify individual module behaviors: `test_preprocessor.py` validates output shape and trapezoidal ROI masking; `test_geometry.py` validates analytical curvature calculations for straight lines (radius > 1000m) and verified zero lateral offset for centered vehicles.
2. **Integration Testing:** `test_pipeline.py` passes synthetic road frames through the full pipeline to verify tensor consistency and assert latency bounds (< 500 ms under debug harness).
3. **Scenario Edge-Case Testing:** Evaluated against 5 distinct scenarios: sharp right curves (curvature ~450 m), sharp left curves (~440 m), close headway hazard (distance < 15 m), empty highway (zero obstacles), and straight cruising.

11. Challenges Faced

- **Perspective Distortion of Road Curvature:** Calibrating trapezoidal source points required careful tuning of the camera horizon line ($y = 420$ px). An incorrect horizon introduced non-linear warping artifacts in the bird's-eye view, destabilizing the polynomial fit. This was resolved by implementing a normalized calibration schema in `default_config.yaml`.
- **CPU Inference Latency Optimization:** Running full deep learning models on CPU frequently resulted in dropped frames. By utilizing the anchor-free YOLOv8 nano variant and vectorizing sliding window pixel extraction using NumPy indexing, per-frame latency was compressed to 50.22 ms, achieving ~20 FPS.
- **False Positive Edge Detections from Road Cracks:** Asphalt road seams and shadows produced spurious Sobel gradient edges. This was mitigated by combining Sobel gradients with HLS/LAB color masks via bitwise logical AND operations, rejecting non-marker textured road artifacts.

12. Learnings and Key Takeaways

This project provided deep practical insights into engineering cyber-physical perception systems. Key takeaways include: (1) the power of fusing classical mathematical optics (homography, calculus) with modern deep neural networks to produce interpretable, robust perception systems; (2) the critical importance of color space transformations beyond RGB for outdoor computer vision; and (3) software engineering principles for real-time edge AI, including modular architecture, configuration decoupling, and automated regression testing.

13. Future Enhancements

- **Temporal Tracking with Kalman Filters:** Integrate an Extended Kalman Filter (EKF) to track lane boundary polynomial states and predict vehicle trajectory during momentary line dropouts.
- **Multi-Lane Detection:** Expand sliding window search to identify adjacent left and right lane boundaries for automated lane change maneuvers.
- **Embedded NPU Deployment:** Quantize model weights to INT8 and compile onto ONNX Runtime or TensorRT for deployment on edge processors (e.g. Raspberry Pi 5 or NVIDIA Jetson).

14. References

- [1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, 'You Only Look Once: Unified, Real-Time Object Detection,' in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.
- [2] Ultralytics, 'YOLOv8 Real-Time Object Detection and Image Segmentation Model,' *Ultralytics Documentation*, 2023. Available: <https://docs.ultralytics.com/>
- [3] M. Aly, 'Real time Detection of Lane Markers in Urban Streets,' in *IEEE Intelligent Vehicles Symposium (IV)*, Eindhoven, Netherlands, 2008, pp. 7–12.
- [4] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge: Cambridge University Press, 2004.
- [5] G. Bradski, 'The OpenCV Library,' *Dr. Dobbs's Journal of Software Tools*, vol. 25, no. 11, pp. 120–125, 2000.
- [6] T. Y. Lin et al., 'Microsoft COCO: Common Objects in Context,' in *European Conference on Computer Vision (ECCV)*, Cham: Springer, 2014, pp. 740–755.